ZTI-MCP

Constraint-Preserving Delegation and Cumulative Effect Authorization

draft-messaoudene-zti-mcp-00

Proposed experimental profile | Revision 0.1 | 31 October 2026

Mohammed Messaoudene Individual contribution

Abstract

This document specifies an experimental security profile for the Model Context Protocol (MCP). It binds delegated agent actions to an immutable mandate, preserves restrictions through delegation, and accounts for consumed and outstanding effects in a shared authorization registry. The profile defines signed mandate and execution tokens, a deterministic cumulative policy gate, and current authorization at the provider's protected effect boundary. An admission token is insufficient on its own: a conforming provider couples revalidation, unique consumption and its declared effect or irrevocable acceptance. The profile addresses cumulative budget violations and explicitly represented forbidden combinations within an administered root scope. It does not infer hidden human intentions, establish complete observation of activities outside that scope, or provide a generic transaction across arbitrary external services.

Editorial status

Individual, unsubmitted Internet-Draft candidate. The document name is proposed, not assigned. No IETF, AAIF, MCP, university or other organizational endorsement is claimed. Intended status describes a proposal, not an approval. Review copy dated 31 October 2026.

The complete specification follows. RFCXML, plain text generated locally with xml2rfc 3.35.0, schemas, worked messages and a bounded implementation report accompany this reading PDF. The XML grammar and processing reports are included. idnits has not been run; no submission or adoption is claimed.

Copyright (c) 2026 Mohammed Messaoudene. The XML source selects standard IETF Trust boilerplate for prospective submission. This review copy has not been submitted.

Contents

1. Introduction
2. Terminology and Data Model
3. ZTI-MCP Architecture and Lifecycle
4. Intent and Constraint Token
5. Deterministic Vectorial Gate
6. Concurrency, Revocation and Failure Semantics
7. Security Considerations and Observability Limits
8. MCP Binding and Interoperability
9. Privacy and Operational Considerations
10. Conformance and Implementation Evidence
11. IANA Considerations
12. Acknowledgements and Contribution Status
13. References
A. Public Laboratory Action and Mandate Policy
B. Worked ICT and Documentary Receipt
C. Bounded R4 Evidence and Reproduction

Reading convention

Uppercase requirements are proposed obligations of ZTI-MCP. References to MCP and RFCs identify reused foundations. Statements about the companion experiment are bounded implementation evidence, not an adopted standard.

1. Introduction

An agent may delegate individually permitted subtasks to other agents or tools while the combined activity violates a shared restriction. An intermediary may also exercise its own credentials without retaining the restrictions of the originating mandate. The protected object in this document is the authority exercised by the resulting actions, rather than the identity or reputation of the language model that proposed them.

This document uses the term compositional constraint evasion for a sequence or concurrent set of actions that satisfies isolated checks but violates a stated cumulative policy. The term is a local description, not a claim that every such case has one established threat taxonomy. A confused-deputy problem is related but distinct: an intermediary can misuse authority even without a cumulative violation.

ZTI-MCP is an application security profile carried over MCP. It does not introduce a universal network router, change MCP transports, or give a gateway control over unmediated resources. The same logical profile can use a gateway, sidecar or integrated execution controller. Physical placement does not remove the provider integration obligations in Section 3.

This revision selects one authorization contract: current-online. Admission creates a revocable reservation. Authority is revalidated at the protected effect boundary, not merely when a request enters the network. Rights acquired permanently at initial admission, offline acceptance, audit-only enforcement and fail-open behavior are not conforming fallback modes.

The word Intent in the profile name refers to declared purpose as contextual input. A signed statement establishes integrity and attribution under a trust configuration; it does not establish the truth of a human narrative. The mandatory gate uses defined costs, restrictions and authoritative facts. A semantic classifier, when used by a deployment, is a separately assessed source of evidence.

1.1. Requirements language and document status

Uppercase requirement terms have the meanings defined by BCP 14 [RFC2119] and [RFC8174]. Lowercase uses are ordinary prose. Every normative requirement in this document is a proposed requirement of this individual profile, not an amendment to the MCP specification or an adopted IETF standard.

The baseline protocol version is MCP 2026-07-28. Other revisions need an explicit binding and a new compatibility review. The identifier zti-mcp/0.1 identifies this profile revision by private agreement. This document is suitable for technical review as a complete individual draft candidate; an official submission and the normal editorial and community review processes remain separate actions.

1.2. Scope and non-goals

A root defines the administered scope of cumulative accounting. Global_Mandate does not mean that the registry observes all activities of a person, all roots, or all organizations. The issuing authority determines which authenticated activities share a root. A caller-selected correlation identifier MUST NOT create a new budget or bypass an existing authority scope.

The base profile covers an immutable root policy, finite non-negative quantities, positive fact labels and monotone forbidden combinations. It does not define policy learning, general natural-language entailment, legal classification, refunds of gross consumed costs, periodic budget renewal, cross-root consensus, or a universal physical outcome detector. These exclusions are necessary to state a testable contract.

2. Terminology and Data Model

Term	Definition
Global_Mandate	An immutable authority record binding a principal, a delegated invoker, a root scope, a validity interval and executable restrictions. A root also owns the cumulative policy.
Execution_Constraint	A restriction whose semantics and enforcement inputs are defined by this profile: an explicit allow-set, a per-action cap, a root budget, a validity restriction or a root forbidden fact set.
Cumulative_State_Registry (CSR)	The authoritative service or transactional state domain that orders mandate, reservation, consumption, cancellation and revocation transitions within a root.
Action_Binding	The SHA-256 commitment to the canonical final action envelope defined in Section 4. It binds what is to be executed, not merely a natural-language task description.
Intent and Constraint Token (ICT)	A short-lived signed assertion identifying one reservation and its precise action, invoker, audience, root, mandate, costs and facts. It is not sufficient authority without current registry validation.
Protected effect boundary	The point at which the resource mutation or explicitly declared irrevocable provider acceptance is authorized and committed. Completion may occur later.
T_initial	The ordered transition that admits and reserves the proposed action. It grants no permission to bypass a later current-state check.
T_revalidate	The authorization decision coupled to consumption and the protected effect boundary. This is not an arbitrary earlier network validation timestamp.
UNKNOWN	A final-outcome uncertainty state after protected acceptance was COMMITTED. Authority remains charged. It is not evidence of cancellation.

2.1. Roles and authenticated identities

The Principal owns the delegated authority. The Proposer is an agent or client that submits an action. The Mandate Authority issues signed records under an established authorization policy. The Gateway controls delivery to a protected downstream tool. The Provider controls the protected effect. The Recorder retains the authoritative transitions and results. Components MAY combine these roles.

[R01] The deployment MUST identify its trusted authority, registry, provider integration and authenticators. Statements from a proposer, clientInfo, tool descriptions or an extension advertisement MUST NOT be used as proof of those trust assumptions. The invoker identity used for authorization MUST come from the applicable authenticated channel or trusted local execution context.

A proxy invokes a downstream resource with the identity and credentials that resource accepts. The ICT sub claim names that authorized invoker. It does not automatically name the human principal, which remains identified in the mandate. If the proxy identity differs from the upstream delegatee, the authority MUST issue an attenuated child mandate naming the proxy before it issues the downstream ICT. Rewriting sub alone is forbidden. A proxy MUST NOT pass an unrelated upstream OAuth token to a downstream resource as a substitute for appropriate downstream authentication [MCP-AUTH].

2.2. Root policy and inherited restrictions

[R02] Each mandate MUST have exactly one parent, except the root. The registry MUST reject cycles, unknown parents, mismatched parent digests and chains longer than eight mandates, including the root. The issuer, tenant, principal, root and authority-assigned correlation identity MUST remain consistent throughout the chain.

The root_policy object contains budgets and forbidden_sets. It is present only on the root; child records contain root_policy: null and the same policy_hash. The base profile has one cumulative budget vector per root. It defines no independently replenished child budget and no child-specific cumulative fact policy.

[R03] A child MUST only narrow allowed_executors, allowed_resources, allowed_methods and allowed_tools by set inclusion, reduce or retain every per-action cap, and use a validity interval contained in its parent interval. It MUST NOT replace the root policy, omit a required restriction, change units, add a budget dimension, or treat an unknown constraint as permissive. An empty allow-set authorizes no matching action.

Checking JSON shape alone is insufficient to establish attenuation. The authority MUST apply these typed comparison rules. It MUST resolve the complete authoritative lineage when reserving and consuming authority. Copying a subset of restrictions into a token does not remove ancestor obligations.

A delegation request MUST be authenticated as the parent delegatee or an explicitly authorized administrator. A new child has a new immutable mandate_id and parent_hash. Delegation, renaming, a new correlation label or another token signature MUST NOT reset root counters. Changes to the root policy or replenishment require a separately defined migration protocol and are outside revision 0.1.

2.3. Quantities, identifiers and canonical interpretation

[R04] Budgets, action caps and cost bounds use a map from dimension name to {unit, amount}. Each map MUST contain exactly the dimensions of the root policy, including dimensions with zero cost. Units MUST match exactly; conversion is outside the base profile. amount, authority_epoch and persistent revisions use canonical unsigned decimal strings in the range 0 through 18446744073709551615. Leading zeros, signs, fractions, exponents and overflow MUST be rejected.

The registry MUST use exact arithmetic or conservative integer checks that cannot create authority by rounding or overflow. Boolean values MUST NOT be accepted as integer values. JWT NumericDate claims and validity fields are JSON integers between 0 and 9007199254740991, with the stricter temporal relationships in Section 4.

Identifiers are case-sensitive opaque strings unless a referenced standard defines them otherwise. Set members MUST be unique. Providers MUST NOT introduce undocumented URI normalization, alias resolution, numeric coercion or wildcard matching. Quantitative dimension names are ASCII strings of 1 to 64 characters using letters, digits, underscore, dot or hyphen.

Within one configured authority domain, Operation_Key is (tenant_id, root_mandate_id, operation_id) and Reservation_Key is (tenant_id, root_mandate_id, reservation_id). The issuer is resolved through that configured authority relationship. Every mutation and lookup MUST validate the complete key and the authenticated caller rights. Identical short IDs in another tenant or root convey no authority.

The Action_Binding and document digests use JCS [RFC8785] and UTF-8 with SHA-256. Duplicate object names, non-finite numbers, invalid Unicode and inputs outside the canonicalizer contract MUST be rejected before binding. JCS property ordering and number serialization MUST NOT be replaced by a superficially similar general-purpose JSON pretty-printer.

3. ZTI-MCP Architecture and Lifecycle

A deployment first establishes the protected resource scope and trusted effect adapters. It then issues a root mandate, registers attenuated descendants, reserves a proposed final action, forwards a valid ICT only on an authorized path, and consumes that reservation at the provider boundary. The registry remains authoritative across transport connections and agent changes.

3.1. Mandate creation and registration

[R05] Root issuance MUST be authorized by an authenticated principal or issuing policy. The issuing authority MUST select the accounting root; it MUST NOT accept a proposer-generated task label as proof of a fresh entitlement. Root creation MUST identify the exact resources, operations, dimensions, units, policy and validity covered.

The immutable zti_mandate object contains profile, tenant_id, principal_id, root_mandate_id, mandate_id, parent_id, parent_hash, correlation_id, delegatee_id, valid_from, valid_until, constraints, root_policy and policy_hash. The root has null parent fields. The constraints object contains exactly allowed_executors, allowed_resources, allowed_methods, allowed_tools and action_caps. Each child references its parent object digest.

The mandate_hash is SHA-256(JCS(zti_mandate)). The policy_hash is SHA-256(JCS(the root root_policy object)). These values are lowercase hexadecimal strings of 64 characters. They are commitments to document content, not to arbitrary JWS signature bytes.

The signed mandate uses the distinct JWS type zti-mandate+jwt, ES256, and claims iss, sub, aud, iat, nbf, exp, jti and zti_mandate. sub equals delegatee_id; aud identifies the configured registry; nbf and exp equal valid_from and valid_until. The registry MUST validate the signed record and its lineage before registration. It MUST NOT accept an ICT or OAuth access token as a mandate.

Mandate verification MUST use the same configured-key and ES256 signing-input rules as the ICT, with exactly alg, kid and the mandate-specific typ in the protected header. It MUST enforce the distinct complete mandate claim schema, reject duplicate or additional members, validate issuer and registry audience, require profile zti-mcp/0.1, safe-integer dates, iat <= nbf < exp, and valid_from < valid_until. The ICT maximum lifetime of 60 seconds does not apply to a mandate; the issuing policy controls mandate lifetime.

Registration of an already known mandate identity with identical immutable content MUST be idempotent and MUST preserve its current inactive status, counters and history. Different content under that identity MUST be refused as a conflict. A still-valid old signature MUST NOT reactivate a revoked mandate or reset its root.

A registry MAY distribute immutable signed mandate records by reference rather than exposing every ancestor to every tool. References MUST resolve only through authenticated configured authority services. Failure to resolve a required ancestor causes suspension or refusal; it does not permit execution with an incomplete chain.

3.2. Action preparation, admission and non-delivery

[R06] The trusted adapter MUST derive a conservative effect footprint from the final effective action, including cost bounds and possible fact labels. A proposer's claimed costs or labels are untrusted hints. Unknown footprint, unknown units, unsupported operations or an inability to establish a reliable bound MUST cause refusal or suspension before the protected effect.

Defaults, transformations and routing that alter the protected operation MUST be resolved before the final binding, or cause a new authorization. Mutable resources MUST be bound to an immutable version or a provider precondition checked at the protected boundary. The implementation MUST identify which

mutable inputs its precondition covers. An unchanged pathname is not evidence that the referenced contents are unchanged.

[R07] Reservation admission MUST atomically validate the current lineage, derive or validate the footprint, evaluate the root gate over consumed and outstanding state, and store the unique reservation. Signing and returning an ICT MUST occur only for a reservation that exists in authoritative state. A lost token response does not justify creating a second reservation for the same operation.

[R08] A gateway claiming non-delivery MUST complete its admission checks before handing the protected action to the downstream application. A rejected action MUST NOT be forwarded through a diagnostic, retry, alternate route or audit path that delivers it to that application. Necessary delivery of minimal decision data to trusted authorization components is part of the declared trust boundary, not prohibited downstream execution.

An ICT identifies reserved authority. It does not promise successful completion, establish the truth of declared purpose, or authorize execution after the mandate has become invalid.

3.3. Provider synchronization barrier

[R09] The provider MUST implement an AtomicEffect boundary that orders current revalidation, unique reservation consumption, accounting and the protected resource transition as one authorization unit. This can be a shared transaction or a provider-coordinated commit mechanism with the same stated guarantees. A remote check followed by an independent execute call does not satisfy this requirement.

The protected transition MAY be a local resource mutation. For a remote or physical provider, it MAY instead be the provider's durable, irrevocable acceptance of the exact operation, but only when the deployment explicitly defines acceptance as the protected boundary. Such acceptance MUST make the operation uniquely accountable, prohibit later unvalidated substitution, and provide stable lookup. It MUST NOT be a cancellable queue entry, an admission ticket, or a local promise to send the operation later.

When acceptance is the protected boundary, subsequent completion is a different event. A revocation ordered after acceptance does not retroactively undo that commitment. This profile does not promise cessation of all physical activity at the instant a revocation request reaches a network interface. A deployment requiring a later boundary MUST provide an enforcement mechanism at that later boundary.

[R10] At AtomicEffect, the provider MUST use the authenticated invoker and final action, validate the ICT and current authoritative state, verify the full lineage, and confirm that the reservation remains usable. It MUST bind the resulting transition to the operation_id and reservation_id. If the provider cannot participate in this coupling, it MUST NOT claim strict ZTI-MCP conformance for that operation.

The abstract AtomicEffect primitive is an integration requirement, not a generic distributed transaction algorithm supplied by a JWT. An implementation report MUST name the actual transaction or commit mechanism, its failure model and the provider acknowledgements it relies on.

3.4. States, cancellation and recovery

State	Meaning	Authority accounting
RESERVED	Admitted, still revocable; no protected effect accepted.	Costs and possible facts remain outstanding.
COMMITTED	Protected effect or declared irrevocable acceptance crossed its boundary.	Reservation charge transfers to consumed authority once.
COMPLETED	The defined final outcome has been established.	Consumed charge and historical facts remain.
UNKNOWN	Final outcome is unresolved after COMMITTED.	Consumed charge and facts remain; no automatic repeat.
CANCELLED / EXPIRED / REVOKED	An unconsumed reservation is authoritatively unusable.	Release only after future consumption has been excluded.

[R11] Only a RESERVED reservation may be cancelled or released under this revision. Cancellation, expiry cleanup or revocation cleanup MUST atomically prevent every later consumption before releasing outstanding authority or facts. If a consumption already won the order, the operation remains charged. A timeout, disconnected socket or cancellation notification is not evidence that this transition occurred.

[R12] COMMITTED, COMPLETED and UNKNOWN authority MUST remain charged under the gross-cost policy of revision 0.1. UNKNOWN facts MUST remain in cumulative evaluation. A provider receipt that only establishes acceptance MUST NOT be presented as proof of completion. Refunds, compensation and periodic replenishment require a different explicit policy and are not implied by these states.

A caller that loses an acknowledgement MUST perform authorized lookup using the same operation identity. Registry uncertainty at the caller is not a new registry state. The registry may still show RESERVED or COMMITTED. If the authoritative result cannot be obtained, the caller MUST suspend further action that would duplicate the operation or assume its budget was released.

Lookup and retry MUST validate tenant and invoker access. A matching operation identity with a different bound action is a conflict. A matching already-committed operation MAY return its existing receipt, but MUST NOT cause a second effect.

4. Intent and Constraint Token

An ICT is a JWS Compact Serialization token using the JWT claim model [RFC7515] [RFC7519]. It is transported as integrity-protected constraint context and MUST NOT replace the access credential required by MCP or by a downstream resource. The base profile deliberately has one cryptographic suite to avoid ambiguous negotiation.

4.1. Protected header and validation context

```
{
  "alg": "ES256",
  "kid": "configured-key-identifier",
  "typ": "zti-ict+jwt"
}
```

[R13] A verifier MUST accept only ES256 for revision 0.1, use a configured P-256 verification key authorized for the issuer and token role, and verify the original received JWS signing input. The signature is the 64-octet R || S encoding required by [RFC7518], not an ASN.1 DER signature. An unrecognized kid, retired key or unavailable trust configuration MUST cause refusal or suspension.

The protected header contains exactly alg, kid and typ. Additional header members, including crit, jku, x5u, jwk and b64, are unsupported in this revision and MUST be rejected. The none algorithm, symmetric-algorithm substitution, an embedded untrusted key and key retrieval from a token-supplied URL MUST NOT be accepted.

The strings zti-ict+jwt and zti-mandate+jwt are private, explicitly agreed type discriminators, not registered media types. Their role-specific validation rules are mutually exclusive through the required typ and claim schema, including when authority and provider services share a host or audience identifier. Verifiers MUST apply those rules, following the separation guidance of [RFC8725]. Algorithm agility requires a new explicitly supported profile revision, not an untrusted header override.

4.2. Required claims

Claim	Required interpretation
iss	Exact configured issuer identifier. It selects a trust relationship, not an arbitrary URL to fetch.
sub	Authenticated invoker expected at the provider. It must equal the mandate delegatee for this operation.
aud	One string identifying the intended provider/executor. Audience arrays are not supported in this profile.
iat, nbf, exp	Safe-integer NumericDate values; iat <= nbf < exp and exp - iat <= 60 seconds for ICTs.
jti	Issuer-unique token identifier. The issuer MUST generate it using at least 128 bits of cryptographically secure randomness or an equivalently collision-resistant controlled allocation. String validation alone cannot attest the generation entropy.
zti	The required profile-specific object described below. No other top-level claims are defined in this revision.

zti member	Meaning
profile, mode	Exactly zti-mcp/0.1 and current-online.
tenant_id, root_mandate_id, mandate_id	Authoritative tenant and mandate references.
mandate_hash, policy_hash	Digests of the immutable mandate and root policy.
correlation_id	Root-assigned task correlation; never sufficient authority by itself.
operation_id, reservation_id	Stable authorized operation and unique reservation identities.
action_hash	Digest of the final Action_Binding envelope.
authority_epoch	Canonical uint64 text for the current root authority epoch at issuance.
cost_bounds	Complete dimension map of authoritative quantity bounds.
facts	Unique positive fact labels describing the conservative possible footprint.

[R14] Every listed claim and member is REQUIRED. A verifier MUST reject unknown top-level or zti members in revision 0.1, duplicate keys, wrong types and absent fields. Comparison of the ICT to its authoritative reservation MUST cover the issuer, subject, audience, tenant_id, root, mandate, correlation_id, digests, epoch, operation, reservation, costs and facts. A valid signature on inconsistent claims is insufficient.

The registry MUST enforce uniqueness of (issuer, jti) and the stable operation/reservation identity independently of the complete token byte string. A new valid ES256 signature over the same claims MUST NOT create a new right. Merely caching a hash of one serialized JWT is insufficient replay control.

[R15] At T_revalidate, nbf <= t < exp MUST hold using the declared authoritative time policy. The ICT interval MUST also fit all mandate validity intervals. An implementation with a bounded clock uncertainty MUST use a conservative interval check; an unbounded or regressing clock MUST cause suspension until trustworthy ordering and time validity are restored. No implicit expiry grace or cached online-valid result is allowed.

The temporal check belongs to the declared authorization decision point. It does not promise that remote completion occurs before exp. Clock recovery and process restart MUST NOT reset epochs, reservations, spent authority or replay identities.

4.3. Action_Binding envelope

The canonical envelope contains exactly profile, tenant_id, root_mandate_id, mandate_id, operation_id, executor_id, resource_id, method, tool_name, tool_revision and arguments. profile is zti-mcp/0.1. executor_id equals the ICT audience. method identifies the requested MCP operation; the base binding in Section 8 covers tools/call. tool_revision identifies the trusted implementation/catalogue contract used to interpret the action.

[R16] action_hash MUST equal lowercase-hex(SHA-256(JCS(envelope))). arguments MUST be the final effective arguments, including required defaults and provider preconditions. An adapter MUST NOT strip or rewrite semantically relevant fields after the binding. A change of executor, resource, tool revision, arguments, tenant, mandate or operation identity requires new authorization.

JSON-RPC request IDs, transport connection IDs and observational tracing headers are excluded because they do not define stable operation identity. All metadata and provider preconditions that change the protected action MUST be included in the envelope, directly or through an immutable reference or digest included in it. The provider MUST resolve and enforce that same committed precondition at AtomicEffect. A mutable out-of-band precondition is insufficient; a field cannot be excluded merely by calling it a header.

The envelope digest prevents undetected substitution under the cryptographic assumptions. It does not prove that an untrusted provider faithfully executes the envelope, that the resource classification is truthful, or that all activity outside the root has been observed.

4.4. Selective disclosure and zero-knowledge proofs

Zero-knowledge proofs and selective-disclosure credentials are not defined by the baseline. Implementations MUST NOT replace the required online registry check with a claim that a general intent proof exists. A future proof binding would need an explicit predicate, witness, issuer trust model, freshness and revocation semantics, action binding, disclosure policy and conformance vectors.

An optional semantic assessment MAY influence whether the authority issues a mandate or reserves an action. Its scores, false positives, false negatives and abstention behavior MUST be evaluated separately from the transactional requirements. The mandatory Vectorial Gate below contains no embedding distance or psychological inference.

5. Deterministic Vectorial Gate

Vectorial denotes a vector of explicitly measured dimensions and a set of authoritative facts. It does not denote an embedding model. Let B be the root budget vector, D the sum of charges consumed at COMMITTED, R the sum of unconsumed active reservations, and F the union of consumed facts and all facts still possible from active reservations.

[R17] Admission MUST evaluate every root dimension componentwise and reject if D[k] + R[k] + c[k] exceeds B[k]. It MUST also reject when any root forbidden_set is a subset of F union candidate_facts. The entire read, evaluation and reservation update MUST be one ordered state transition. Checking each action in isolation is non-conforming.

A forbidden_set is a nonempty set of unique positive labels. Every member must be in the deployment's authoritative fact vocabulary. The root predicate is monotone: adding facts cannot turn a forbidden state into a permitted state. Removing a cancelled reservation's possible facts is permitted only under the safe cancellation rule. Historical consumed facts are never erased in revision 0.1.

```
RESERVE(authenticated_invoker, envelope):
  begin ordered transaction for envelope.root_mandate_id
  resolve and validate current complete mandate lineage
  require invoker == current mandate.delegatee_id
  reject duplicate operation with different envelope digest
  return existing reservation/result for an identical retry
  derive trusted costs c and possible facts f
  enforce all ancestor allow-sets, per-action caps and validity
  read D, R and F from authoritative root state
  require for every k: D[k] + R[k] + c[k] <= B[k]
  require no forbidden_set is a subset of (F union f)
  create RESERVED record bound to envelope, c, f and identity
  commit reservation transition
  issue role-specific ICT for that stored reservation
```

The gate treats an outstanding reservation as a possible future effect. If action A reserves fact sensitive_read, a second action whose possible facts contain external_export is rejected when the root prohibits {sensitive_read, external_export}, even when A has not yet committed. This is a conservative policy example, not a statement that every sensitive read or every export is intrinsically prohibited.

The labels must be obtained from a trusted catalogue, versioned resources or equivalent authoritative provider evidence. The system cannot establish this guarantee if agents freely choose or omit their own labels. False classifications and incomplete catalogues are explicit limitations of the deployment.

5.1. Revalidation and atomic consumption

```
ATOMIC_EFFECT(authenticated_invoker, ict, final_envelope):
  validate JWS type, signature, fixed issuer and recipient
  enter the provider/registry protected authorization unit
  load the exact authoritative operation and reservation
  validate current root and complete ancestor status
  require matching epoch, identity, action and constraints
  require current time validity and usable RESERVED state
  recheck authoritative footprint and provider preconditions
  validate the root invariant including this reservation once
  atomically:
    make the protected effect or irrevocable acceptance
    change RESERVED to COMMITTED exactly once
    transfer its charge from R to D and retain its facts
    persist the operation receipt and state revision
  return the durable receipt or an unresolved-ack indication
```

[R18] Revalidation MUST NOT add its own existing reservation a second time. Consuming it transfers authority from R to D while preserving D + R. A changed footprint MUST be rejected and newly authorized before effect; a provider MUST NOT execute first and retroactively increase the charge to make a violation appear compliant.

If the protected operation fails before the atomic boundary, the implementation can retain or safely cancel its reservation. If it may have crossed the boundary, recovery MUST use authoritative lookup and preserve its possible or consumed charge. Infrastructure failure MUST NOT be counted as a successful policy refusal.

A receipt MUST contain profile, tenant_id, root_mandate_id, mandate_id, operation_id, reservation_id, action_hash, effect_boundary, status and state_revision. profile is zti-mcp/0.1; effect_boundary is resource-mutation or provider-acceptance as fixed by the trusted provider contract; status is COMMITTED, COMPLETED or UNKNOWN. The other fields use their previously defined types. A receipt MUST be delivered over an authenticated channel or have a separately specified authenticated envelope. It records the claimed boundary outcome, not a universal proof of external completion.

5.2. Conditional safety argument

Assume a reliable root registry, complete mediation, truthful conservative footprints, exact non-negative arithmetic and unique consumption. Initially D = R = 0. Admission adds c only when D + R + c <= B. Consumption transfers c from R to D. Safe cancellation removes only an unconsumed reservation that cannot later be exercised. Consequently D + R <= B is preserved by these transitions.

For forbidden sets, admission checks the union containing all possible reserved effects. Consumption does not introduce a fact outside that reserved set. Safe cancellation only removes effects made impossible. A forbidden positive combination therefore cannot appear through a conforming admitted transition under the footprint assumptions.

These are conditional invariants of the stated transition system. They do not prove a classifier correct, infer unknown activities, establish a global real-world intent property or demonstrate that a remote provider implements AtomicEffect. The experiment and implementation report must be evaluated separately.

6. Concurrency, Revocation and Failure Semantics

[R19] The registry MUST provide a linearizable order for conflicting transitions within a root, including reservation, cancellation, root or ancestor revocation, consumption and administrative status changes. A set of eventually consistent counters or independent per-agent balances does not satisfy this requirement. Implementations MAY shard independent roots but MUST NOT claim a shared bound across those roots without an additional authority policy.

The profile does not choose a consensus algorithm. A deployment report MUST specify the storage and failover mechanism that preserves the required order. A replicated implementation that cannot establish current authority during a partition MUST suspend protected effects rather than silently accept a cached positive decision.

6.1. Revocation ordering

[R20] Root revocation MUST make the root inactive and advance its authority epoch in the same ordered state domain used by consumption. At the maximum representable epoch, it MUST instead make the root terminally inactive without wrapping to zero; saturation MUST NOT leave revoked authority usable. Ancestor revocation MUST make that ancestor inactive so that every descendant consumption fails its live lineage check. An ICT epoch match alone is not a substitute for checking revoked intermediate mandates.

The authority epoch changes on an authority decision, not on ordinary reservation, consumption or cancellation bookkeeping. Those transitions update the state revision and are still read atomically. If a bounded state revision saturates, the implementation MUST suspend new protected transitions and retain a terminal inactive or equivalent safe representation for revocation, rather than wrap or reset its history.

The revocation operation MUST return an authenticated acknowledgement identifying the scope and authoritative revision at which it became effective. No consumption ordered after that revocation may use the revoked authority. A request merely sent to a network endpoint is not the revocation linearization point.

If consumption was ordered first, the protected effect or irrevocable acceptance may already exist. Its completion may continue after the revocation acknowledgement. Cancellation of accepted external work, if available, is a separate provider operation and does not erase historical consumed authority.

This is the precise meaning of immediate revocation in the profile: it takes effect in the common authority order without accepting subsequent stale consumption. It is not a zero-latency physical broadcast or retroactive reversal of committed effects.

6.2. Replay, uncertain acknowledgements and durable state

[R21] A stable operation identity MUST survive transport retries, token resigning, process restarts and acknowledgement loss. The registry MUST prevent a second protected effect under a consumed reservation. It MUST refuse operation identity reuse with a different action. Authorized lookup MAY return an already committed result without re-execution.

The implementation MUST retain replay and accounting records for as long as an operation can still be retried, observed or completed under its provider contract. Deletion or compaction MUST preserve the safety-relevant tombstone or equivalent durable fact. The base profile does not define safe reclamation for arbitrary providers.

A trusted time failure, unavailable registry, unresolved mandatory ancestor, unavailable key or unknown effect footprint produces SUSPENDED or a policy-specific refusal without effect. A user-visible timeout MUST NOT be represented as a verified refusal when an effect may already have been accepted.

Availability is intentionally subordinate to the stated authorization property during loss of current authority. Operators SHOULD bound queue sizes, pending reservations and resource consumption, and SHOULD provide authenticated resolution and appeal mechanisms for prolonged suspension.

7. Security Considerations and Observability Limits

The adversary may control a proposer, fragment a task among agents, omit claimed context, replay tokens, substitute arguments or destinations, attempt to widen a child mandate, race simultaneous reservations, choose misleading task labels, exploit timeouts or try to omit the profile. The adversary may also provide hostile tool output or policy-looking natural language. Such material is untrusted data.

[R22] The implementation MUST prevent access to protected effects through unmediated alternate paths under the same claimed scope. Administrative access, direct database writes and provider backdoors are outside the stated guarantee unless the deployment actually mediates them. It MUST disclose these exclusions rather than implying network-wide prevention.

The authority and enforcement adapters are trusted to execute their contract. Compromise of their signing keys, registry state or resource classification can defeat the profile. Key rotation, revocation and incident response SHOULD be supported, but possession of a signature does not prove that the signer is uncompromised.

Unique consumption prevents a second effect under the same Operation_Key. It does not automatically recognize that two newly authorized operation identities describe the same real-world intention. Semantic deduplication and provider business identifiers require their own policy. A caller MUST NOT evade unresolved-outcome recovery simply by allocating a fresh operation identity.

Open-weight and proprietary models are treated identically as proposers. The profile does not assume either category is intrinsically safe or malicious. A local model can still act on resources it controls outside the administered perimeter; ZTI-MCP cannot remove that authority by naming a protocol.

Action digests do not authenticate semantics. Incorrect tool catalogues, mutable resources without checked preconditions, undocumented defaults, response-dependent hidden actions and incomplete effect footprints can invalidate the assumptions. Multi-stage tools MUST expose separately protected boundaries or reserve a conservative footprint for the complete operation.

A signed, incomplete trace remains incomplete. A correlation identifier does not prove that every related activity belongs to the observed root. Two independently granted roots can collectively exceed a desired global limit while each respects its own policy. Root issuance and cross-root allocation require separate governance.

The profile may conservatively refuse a harmless operation because another reservation could combine with it. Unknown labels and suspended state can reduce availability. These consequences MUST be reported separately from successful detection of an actual harmful purpose. A policy or classifier error is not repaired by signing its result.

The predicate language in revision 0.1 intentionally uses finite positive labels and monotone forbidden sets. Arbitrary executable policies, negative facts, sequence-dependent obligations, probabilistic risk thresholds and learned embeddings are outside the mandatory gate. An extension defining them would require new semantics and evidence.

[R23] Parsers and verifiers MUST impose explicit input size, nesting, lineage and pending-state limits. They MUST reject duplicate members and unsupported mandatory values. They MUST NOT fetch issuer URLs, key URLs, schema references or registry endpoints supplied by an untrusted token without an authenticated configured trust relationship.

No statement in this document establishes criminal intent, legal compliance, liability immunity, patentability or a universally safe assembly of actions. The specified properties concern the declared policy, observed scope and trusted integration.

8. MCP Binding and Interoperability

This binding targets MCP 2026-07-28 [MCP-BASE] [MCP-EXT]. It uses the existing extension and metadata mechanism. EXT_ID is a deployment-agreed identifier of the form <controlled-reverse-DNS-prefix>/zti-mcp. The value org.example/zti-mcp in examples is reserved documentation space [RFC2606], not an assigned production identifier.

[R24] Production participants MUST agree on an EXT_ID whose prefix is controlled by the deploying party and is not reserved to MCP. They MUST NOT use io.modelcontextprotocol or another reserved prefix for this individual proposal. Different configured EXT_ID values are not automatically interoperable, even when both advertise zti-mcp/0.1.

8.1. Discovery and per-request requirements

The server advertises the extension in result.capabilities.extensions[EXT_ID] returned by server/discover. The client supplies its relevant extension capability in every protected request under params._meta["io.modelcontextprotocol/clientCapabilities"].extensions[EXT_ID]. Both advertise revision "0.1", profile "zti-mcp/0.1", mode "current-online" and effect_coupling "required".

The action metadata at params._meta[EXT_ID] contains exactly revision "0.1", operation_id and ict. The per-request MCP protocolVersion and clientCapabilities fields remain required. The operation_id MUST match the token and the final bound action. Missing extension support or a different revision MUST NOT trigger an admission-only fallback.

[R25] The provider MUST verify both protocol compatibility and the actual enforcement path. An advertised effect_coupling value is a capability claim, not attestation that coupling exists. server/discover and the authorized control methods below do not require an execution ICT; their applicable authentication and per-method authority remain required. Other bootstrap exceptions MUST be explicitly enumerated and MUST NOT allow the protected resource transition.

The initial binding protects tools/call operations designated by the deployment. resources/read, sampling, elicitation and additional methods need an explicit effect mapping before strict coverage is claimed. Their existence in an MCP server does not automatically make them ZTI-MCP-protected.

8.2. Registry control methods

The authority service MAY be co-hosted with a provider or separately deployed at a configured authenticated MCP endpoint. For wire interoperability using this binding, it MUST implement the five additive extension methods below. The method name is EXT_ID followed by a dot and the verb; for example org.example/zti-mcp.reserve in laboratory messages. These are proposed private extension methods, not MCP core verbs. Existing core methods MUST NOT be replaced. The official SDK extension mechanism provides an additive method surface [MCP-SDK-EXT].

All control requests carry the pinned MCP protocolVersion, relevant clientCapabilities and agreed extension revision. Their params contain the listed members plus the standard _meta object; unknown control members are rejected. They do not require an ICT, because they create or resolve authority rather than execute the protected business action. Every method still requires the specified authenticated authorization. The endpoint and issuer relationship are configured by the deployment, never discovered from an untrusted JWT URL.

Verb and params	Authorized actor	Required result members beyond resultType
.register {mandate}	Authority role permitted to register the root, or parent delegatee/administrator permitted to register that attenuated child.	status, tenant_id, root_mandate_id, mandate_id, mandate_hash, policy_hash, state_revision.
.reserve {action}	Authenticated delegatee of the leaf mandate named in the action.	status, tenant_id, root_mandate_id, operation_id, reservation_id, state_revision; ict for usable RESERVED authority, receipt for consumed authority.
.lookup {tenant_id, root_mandate_id, operation_id}	Recorded invoker or explicitly authorized tenant administrator.	status, tenant_id, root_mandate_id, operation_id, reservation_id, state_revision; receipt when the protected boundary was crossed.
.cancel {tenant_id, root_mandate_id, operation_id, reservation_id}	Recorded invoker or explicitly authorized tenant administrator.	status, tenant_id, root_mandate_id, operation_id, reservation_id, state_revision; receipt if already consumed. Only CANCELLED releases authority.
.revoke {tenant_id, root_mandate_id, mandate_id}	Authority role explicitly permitted to revoke the identified root or ancestor.	status REVOKED, tenant_id, root_mandate_id, mandate_id, authority_epoch, state_revision.

[R28] Authentication alone MUST NOT authorize a control mutation. Registering a child MUST verify the parent-delegatee authorization and all attenuation rules in addition to the signature. Knowledge of an identifier or possession of a publicly readable signed mandate MUST NOT grant cancel, lookup or revoke access. Unauthorized and nonexistent-object replies SHOULD avoid disclosing another tenant's state.

The .register mandate member is a compact signed mandate JWS. The .reserve action member is the exact envelope of Section 4.3; the registry derives its footprint rather than accepting client cost or fact claims. Successful control results use resultType "complete". Register returns status ACTIVE or the

existing REVOKED state without reactivation. Reserve returns RESERVED, or the existing COMMITTED, COMPLETED, UNKNOWN, CANCELLED, EXPIRED or REVOKED state for an authorized identical retry. A different bound action is OPERATION_CONFLICT.

Every control result with status COMMITTED, COMPLETED or UNKNOWN MUST include receipt, with exactly the object defined in Section 5.1. Other control statuses omit receipt. The ict member is omitted, not null, unless the response returns a usable RESERVED token; an expired token MUST NOT be reissued as usable. Successful result bodies contain exactly their defined members and the standard MCP _meta object when present. No additional resultType value is introduced.

Lookup returns the recorded state without changing it. A request to cancel an already consumed operation MUST return its unchanged state or ALREADY_CONSUMED; it MUST NOT report CANCELLED. Revocation acknowledgement establishes the ordered scope change, not completion of external compensation. All state_revision and authority_epoch values use the canonical uint64 text type.

This binding deliberately defines no public .commit method that grants a transferable decision for arbitrary later execution. AtomicEffect is the provider integration boundary: the protected tools/call invokes that mechanism with its actual command and authenticated invoker. A distributed provider MAY use additional private coordination messages, but MUST document and demonstrate the coupling they implement.

The profile does not define ICT refresh. An expired unconsumed reservation may be safely cancelled through the authoritative transition before a separately authorized new operation is created. An application MUST NOT assume that merely obtaining a new token or changing an operation ID resolves an uncertain accepted operation.

8.3. Transport mapping and provider integration

For Streamable HTTP, the normal Authorization credential remains separate from the ICT. Required MCP-Protocol-Version, Mcp-Method and, for tools/call, Mcp-Name headers MUST match the corresponding message data as required by the pinned MCP transport specification [MCP-HTTP]. The profile defines no parallel HTTP header carrying the entire mandate or history.

For stdio, the same JSON metadata is used. The host MUST establish an authenticated local invocation identity and protect the provider/registry channel against substitution. The existence of a long-lived process or connection MUST NOT stand in for a root, task or authenticated principal. The HTTP OAuth procedure is not automatically imported into stdio [MCP-AUTH].

A gateway MAY mediate both MCP and non-MCP downstream providers, but each provider binding MUST preserve the final action and satisfy AtomicEffect. Wrapping an arbitrary HTTP API with MCP does not create atomic authorization. Unsupported providers MUST be excluded from strict coverage or rejected for protected effects.

8.4. Errors and outcome representation

Existing MCP errors retain their defined meanings: malformed required metadata uses -32602, unsupported MCP protocol version uses -32022, missing required client capability uses -32021, and HTTP/body metadata mismatch uses -32020. Implementations MUST NOT appropriate reserved MCP error codes for unrelated ZTI outcomes.

This experimental profile uses the application-defined JSON-RPC error code -31001 with data.profile = "zti-mcp/0.1" and a reason from the table below. This code is outside the JSON-RPC reserved range. It is a private binding value, not an IANA or MCP assignment; peers MUST scope its interpretation to the negotiated EXT_ID and profile.

Reason	Meaning
NOT_AUTHORIZED / NOT_FOUND / MANDATE_CONFLICT	The authenticated actor lacks the required right, the accessible object is absent, or an immutable mandate identity conflicts.
INVALID_TOKEN / BINDING_MISMATCH	Cryptographic, claim or exact-action validation failed.
CONSTRAINT_VIOLATION / CUMULATIVE_VIOLATION	An inherited action rule or root cumulative predicate failed.
REVOKED / EXPIRED	Current authority or time validity does not permit consumption.
OPERATION_CONFLICT / ALREADY_CONSUMED	Operation mismatch or attempted second consumption.
AUTHORITY_UNAVAILABLE / FOOTPRINT_UNKNOWN	Necessary current state or authoritative effect bound is unavailable.
EFFECT_COUPLING_UNSUPPORTED / PROFILE_UNSUPPORTED	The implementation cannot apply the required strict contract.

Each slash-separated label in the table denotes separate reason values, not a literal value containing a slash. A ZTI-specific -31001 error response MUST put profile, reason and status REFUSED or SUSPENDED in error.data. The optional error.data.operation_ref is exactly {tenant_id, root_mandate_id, operation_id} and is present only when the caller may access that operation. No other data members are defined. Native MCP errors retain their native schemas. Error data MUST NOT reveal unrelated tenants, secret policy contents or token material. A committed operation with an unknown final result MUST be represented as an outcome with status UNKNOWN, not a claim that it was refused. A successful protected tools/call result uses resultType "complete" and carries the ZTI receipt in result._meta[EXT_ID]. Control results use the separate schemas in Section 8.2.

8.5. Relationship to Interceptors and adjacent work

Interceptors is informative prior work, not a dependency. The comparison in this draft is pinned to SEP-2624 at commit 3a2760642010e30037231ef4d7586750b0654e65. Its receiving sequence places validation before mutation, and its execution options include audit and fail-open behavior. An integration claiming this profile therefore needs final-action verification after relevant transformations and mandatory blocking at the protected boundary [INTERCEPTORS].

The atomicity of an interceptor mutation chain does not establish atomicity with a provider effect or shared registry. The required ZTI final gate is an additional integration contract; this document does not claim that the examined proposal already specifies it. An invoker override MUST NOT disable that gate while retaining strict conformance.

OAuth Rich Authorization Requests describes structured rights [RFC9396], while OAuth Token Exchange defines subject/actor relationships without automatically consuming input authority or propagating all revocation [RFC8693]. UCAN separates delegation and invocation and includes attenuation and execution-time validation [UCAN]. Verifiable Intent defines signed commercial mandates and cumulative budget tracking [VI]. These mechanisms are relevant antecedents. This draft claims neither invention of signed mandates nor first use of cumulative authorization.

9. Privacy and Operational Considerations

[R26] A deployment MUST define the data it records, the parties allowed to query it, retention limits and deletion rules that preserve safety-relevant state. Mandate chains and cumulative facts can reveal relationships and sensitive activities. Access to registry lookup and audit results MUST be authorized per tenant and principal.

Providers SHOULD receive only the information needed for their contract. Hash commitments SHOULD be treated as potentially identifying data: a hash of a predictable value does not conceal that value against guessing. Logs SHOULD exclude raw OAuth credentials, ICTs and sensitive arguments when a protected reference suffices.

Signed tokens provide integrity rather than secrecy. Confidentiality of transport and stored state depends on the authenticated channel and storage controls. A shared registry is a concentration of operational and privacy risk; deployments SHOULD document administrative access, failure handling, monitoring, backup and incident procedures.

Operational reporting SHOULD separate invalid requests, genuine policy refusals, unknown footprints, infrastructure suspensions, committed actions and uncertain outcomes. Counting every timeout as a prevented attack would misrepresent the system.

10. Conformance and Implementation Evidence

[R27] A conformance claim MUST name the profile revision, MCP binding, protected operations, issuer trust configuration, authoritative footprint source, registry ordering mechanism, time policy, provider effect boundary and recovery behavior. Support for parsing an ICT or advertising an extension is not sufficient.

• Positive execution: a permitted action must produce the declared protected transition and a corresponding durable receipt.

• Attenuation: a child cannot widen an ancestor restriction, change a unit, reset a root or replace a cumulative policy.

• Concurrency: two individually valid proposals cannot jointly exceed a shared dimension or create a forbidden combination through outstanding reservations.

• Current authority: root and intermediate revocation, expiry and changed preconditions prevent later consumption in the defined order.

• Binding and replay: changes to action, identity, audience or scope fail; resigning identical claims cannot produce a second effect.

• Recovery: acknowledgement loss, cancellation races and UNKNOWN preserve the correct single charge and possible facts.

• Compatibility: missing profile, unsupported revision, unavailable authority and disabled final enforcement do not silently weaken the contract.

The R4 companion experiment is a bounded local implementation, described in an implementation report shipped with this draft. It is not the normative specification. It uses local SQLite for the tested effect boundary, signed fixture credentials and a restricted canonicalization corpus. Its exact test counts, mutations, source digests and observed failures are recorded in that report, rather than inferred from the normative requirements.

The local tests do not establish production MCP SDK interoperability, full JCS coverage, external-provider atomicity, complete crash recovery, distributed consensus, classifier accuracy or independent institutional certification. A public maturity claim requires a provider binding, at least one genuinely independent implementation and a reviewed interoperability corpus covering the selected profile.

11. IANA Considerations

This individual experimental draft requests no IANA actions. The JWT claims zti and zti_mandate and the type discriminators zti-ict+jwt and zti-mandate+jwt are private values used by explicit agreement. No new registered media type, OAuth parameter, URI scheme or IANA-assigned error code is claimed.

The metadata identifier EXT_ID is a deployment-controlled MCP extension identifier. org.example/zti-mcp is illustrative reserved example space and MUST NOT be advertised as an assigned production namespace. A future proposal for public registration or official MCP extension status requires its own registration text and review.

12. Acknowledgements and Contribution Status

Mohammed Messaoudene initiated and directed the problem formulation. AI assistance was used for drafting, source review, implementation and cross-review. The review roles belong to one assisted workflow and are not independent human or institutional certification.

This draft preserves the bounded lessons of the preceding R1-R3 studies while selecting the stricter current-online contract. Previous code and results remain separate. No public submission, registry allocation, institutional sponsorship or approval has been performed by creating this document.

13. References

13.1. Normative References

[RFC2119] Key words for use in RFCs to Indicate Requirement Levels. March 1997.

https://www.rfc-editor.org/rfc/rfc2119

[RFC8174] Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. May 2017.

https://www.rfc-editor.org/rfc/rfc8174

[RFC7515] JSON Web Signature (JWS). May 2015.

https://www.rfc-editor.org/rfc/rfc7515

[RFC7518] JSON Web Algorithms (JWA). May 2015.

https://www.rfc-editor.org/rfc/rfc7518

[RFC7519] JSON Web Token (JWT). May 2015.

https://www.rfc-editor.org/rfc/rfc7519

[RFC8725] JSON Web Token Best Current Practices. February 2020.

https://www.rfc-editor.org/rfc/rfc8725

[RFC8785] JSON Canonicalization Scheme (JCS). June 2020.

https://www.rfc-editor.org/rfc/rfc8785

[RFC2606] Reserved Top Level DNS Names. June 1999.

https://www.rfc-editor.org/rfc/rfc2606

[MCP-BASE] MCP Base Protocol, revision 2026-07-28. 2026-07-28.

https://modelcontextprotocol.io/specification/2026-07-28/basic

[MCP-EXT] MCP Extensions Overview, accessed 2026-10-31. 2026-10-31.

https://modelcontextprotocol.io/extensions/overview

[MCP-HTTP] MCP Streamable HTTP, revision 2026-07-28. 2026-07-28.

https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http

[MCP-AUTH] MCP Authorization, revision 2026-07-28. 2026-07-28.

https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization

13.2. Informative References

[MCP-SDK-EXT] Official Python MCP SDK: Extensions, inspected 2026-10-31. 2026-10-31.

https://py.sdk.modelcontextprotocol.io/advanced/extensions/

[RFC9396] OAuth 2.0 Rich Authorization Requests. May 2023.

https://www.rfc-editor.org/rfc/rfc9396

[RFC8693] OAuth 2.0 Token Exchange. January 2020.

https://www.rfc-editor.org/rfc/rfc8693

[INTERCEPTORS] SEP-2624, Interceptors for MCP, Draft; revision 3a2760642010e30037231ef4d7586750b0654e65. July 2026.

https://github.com/Degiorgio/modelcontextprotocol/blob/3a2760642010e30037231ef4d7586750b0654e65/seps/2624-interceptors-for-model-context-protocol.md

[UCAN] UCAN Specification, v1.0.0 observed 2026-10-31. 2026-10-31.

https://github.com/ucan-wg/spec

[VI] Verifiable Intent Specification Overview, 0.1-draft. February 2026.

https://verifiableintent.dev/spec/

A. Public Laboratory Action and Mandate Policy

This appendix is informative. The action and policy below are exported from the successful C02 narrowed-child scenario in the companion experiment. All identifiers refer to public laboratory fixtures. The fixture clock, public test signing material and example domains are not live credentials or production configuration.

The final Action_Binding envelope is the exact object hashed by the exported ICT. Its method is tools/call. The hash does not include transport headers or the JSON-RPC request identifier.


```
{
  "arguments": {
    "payload": "",
    "units": "1"
  },
  "executor_id": "urn:lab:executor",
  "mandate_id": "child-964917085ccc1da032f2b3d63fbcc2db",
  "method": "tools/call",
  "operation_id": "operation-1",
  "profile": "zti-mcp/0.1",
  "resource_id": "urn:lab:resource",
  "root_mandate_id": "root-596ad77dd6298ee578c1077594e869f7",
  "tenant_id": "lab-tenant",
  "tool_name": "lab.compute",
  "tool_revision": "1"
}
```


The root owns the following cumulative policy. The child retains the same policy_hash and root identity while narrowing the allowed tools and per-action bounds. It carries root_policy: null; that value does not remove the root policy.


```
{
  "budgets": {
    "bytes": {
      "amount": "10",
      "unit": "byte"
    },
    "ops": {
      "amount": "3",
      "unit": "operation"
    }
  },
  "forbidden_sets": [
    [
      "read_sensitive",
      "export_external"
    ]
  ]
}
```

The narrowed child constraints used in C02 are:

```
{
  "action_caps": {
    "bytes": {
      "amount": "5",
      "unit": "byte"
    },
    "ops": {
      "amount": "1",
      "unit": "operation"
    }
  },
  "allowed_executors": [
    "urn:lab:executor"
  ],
  "allowed_methods": [
    "tools/call"
  ],
  "allowed_resources": [
    "urn:lab:resource"
  ],
  "allowed_tools": [
    "lab.compute"
  ]
}
```

Complete signed root and child mandates, decoded claims, the public JWK and the verification record are included in examples/. The test private key is deliberately public laboratory material and cannot be used as a production trust anchor.

B. Worked ICT and Documentary Receipt

The protected JWS header and decoded claims below correspond exactly to examples/ict.jwt. That file contains the complete compact serialization without abbreviated signature bytes. Its ES256 signature, the root signature and the child signature were checked using the exported public JWK. These checks establish only the laboratory signature and binding relationships, not current production authorization.

```
{
  "alg": "ES256",
  "kid": "public-laboratory-p256-key",
  "typ": "zti-ict+jwt"
}
```

```
{
  "aud": "urn:lab:executor",
  "exp": 1793404860,
  "iat": 1793404800,
  "iss": "https://authority.example.invalid",
  "jti": "ict-e1864e019f191d6a3fc9a5f1e3411d6b",
  "nbf": 1793404800,
  "sub": "agent-b",
  "zti": {
    "action_hash":
  "ac7ef8806238fbe1b9a156927e98701772d068adfd04ba4e59340c1f11550611",
    "authority_epoch": "0",
    "correlation_id": "corr-b63802c8b203a7d92fe8580f0f157b21",
    "cost_bounds": {
      "bytes": {
        "amount": "0",
        "unit": "byte"
      },
      "ops": {
        "amount": "1",
        "unit": "operation"
      }
    },
    "facts": [],
    "mandate_hash":
  "fea93f16825c413fa08e8d458f45fea58beb50dc400904adc385326258467ed5",
    "mandate_id": "child-964917085ccc1da032f2b3d63fbcc2db",
    "mode": "current-online",
    "operation_id": "operation-1",
    "policy_hash":
  "32cdc6266f90955706728bccf9c2f1d3680749154de671ac219ea4d297f3b90c",
    "profile": "zti-mcp/0.1",
    "reservation_id": "r-7ac035f3202b180f33db64aa6875ee01",
    "root_mandate_id": "root-596ad77dd6298ee578c1077594e869f7",
    "tenant_id": "lab-tenant"
  }
}
```


The following receipt is an offline documentary projection of the persisted C02 COMMITTED effect and audit sequence. It illustrates the required receipt fields. The local Python engine did not emit this complete object over an authenticated MCP channel. The value state_revision is taken from the stored commit audit entry.


```
{
  "action_hash":
  "ac7ef8806238fbe1b9a156927e98701772d068adfd04ba4e59340c1f11550611",
  "effect_boundary": "resource-mutation",
  "mandate_id": "child-964917085ccc1da032f2b3d63fbcc2db",
  "operation_id": "operation-1",
  "profile": "zti-mcp/0.1",
  "reservation_id": "r-7ac035f3202b180f33db64aa6875ee01",
  "root_mandate_id": "root-596ad77dd6298ee578c1077594e869f7",
  "state_revision": "2",
  "status": "COMMITTED",
  "tenant_id": "lab-tenant"
}
```


The examples/wire directory assembles illustrative MCP messages around the signed fixtures, including extension discovery, control calls and a protected tools/call. Those messages were not exchanged between independent MCP implementations. A reader must not infer transport interoperability or successful network negotiation from their presence.

C. Bounded R4 Evidence and Reproduction

The companion R4 experiment implements one local current-online authority engine. The contract, case expectations and interpretation note were recorded before the first full run. The coordinator then reproduced the complete run from the same executable source in a separate output directory. The two campaigns cover the same forty scenarios; they are not eighty distinct scenarios.

Observation	Scope
40 / 40 expected observations per campaign	Forty predefined cases covering positive execution, attenuation, exact quantities, cumulative budgets and facts, current authority, replay and selected recovery states.
3 / 3 targeted mutants detected per campaign	Omitted ancestor checks; ignored outstanding facts; released UNKNOWN charge. These are three deliberately introduced faults, not a general detection rate.
13 child processes per campaign	Real process activity across the concurrent and ordered scenarios; not thirteen simultaneous participants.
0 operational errors reported	The result is separate from policy refusals. It does not measure latency, throughput or availability under deployment load.
43 SQLite databases independently read back	One AI review role reconstructed the initial campaign state without importing the authority engine: forty normal cases and three mutants.
7 targeted correction probes matched	Separate predeclared checks of saturation, inconsistent stored facts, unknown states and empty-value handling. They are additional probes, not additions to the forty-case campaign.

C20, C23 and C37 release two admission processes from a common barrier against shared SQLite. C25 and C26 deliberately order cancellation and consumption with events; they establish the two selected orders and do not measure simultaneous cancellation/consumption lock contention. Traces contain PIDs, worker messages, barriers, exit codes and audit order, but no per-worker timestamps.

The outstanding-fact mutant admits two incompatible reservations. The later consumption gate still refuses one effect: the observed defect is a violation of the possible-fact invariant, not two incompatible committed effects. The UNKNOWN-release mutant produces two effects with gross charge four under budget three. The omitted-ancestor mutant admits an effect under an overbroad child.

The tested AtomicEffect is a SQLite transaction containing the protected local effect row, current checks, accounting and state transition. It is not a transaction across a remote provider. UNKNOWN is a controlled post-commit bookkeeping state in this fixture. Loss of acknowledgement is tested by a real child that commits and omits its response; authoritative lookup retrieves the persisted COMMITTED result.

The canonicalizer covers the declared ASCII-key and safe-integer subset used by these fixtures. Full RFC 8785 conformance is not claimed for that executable. The schemas are structural candidates; JSON syntax and local references were checked, but a complete JSON Schema validator was not available. Signature validation does not validate live authorization state.

The local API is not an implementation of the complete wire profile. It does not provide all external registration ACLs, response schemas, authenticated MCP channels, full production key lifecycle or an independent SDK peer. In particular, its internal result representation must not be exposed unchanged as a conforming control endpoint. The coverage matrix identifies such gaps.

Reproduction uses Python 3.12 with the installed cryptography library and SQLite. From experiment/, run the following command into a fresh directory; existing evidence directories are not overwritten.

```
python3 -I run_r4.py --output-dir runs/new_reproduction
```

The dossier contains source hashes, fixed input hashes, per-case results, SQLite state, the mutation witnesses, correction history, observation errata and review reports. AI-assisted cross-review within one workflow is disclosed; it is not independent human, academic, IETF, AAIF or institutional certification.

Author's Address

Mohammed Messaoudene Individual contribution, Algeria
mohammed.messaoudene@univ-temouchent.edu.dz